

Подход с использованием нечетких решеток Галуа

Выявление требований к облачным сервисам

Ирина Тодоран Койц , Студент-член IEEE и Мартин Глинц, член компьютерного общества IEEE

Аннотация. В последние годы облачная парадигма становится всё более привлекательной благодаря её ожидаемому техническому и экономическому влиянию. Поэтому внимание исследователей и практиков было приковано к улучшению технологических характеристик облачных сервисов, таких как производительность, масштабируемость и безопасность. Однако проблема выявления и понимания реальных потребностей потребителей облачных услуг в значительной степени игнорировалась. Существующие методы выявления требований не подходят для сферы облачных вычислений, где потребители крайне неоднородны и географически распределены, часто запрашивают изменения и ожидают быстрого предоставления услуг. В данной статье мы представляем новый подход к выявлению требований к облачным сервисам, который использует расширенные поисковые запросы потребителей для определения требований, которые могут привести к созданию новых облачных решений. Для этого, исходя из запросов, мы строим нечёткие решётки Галуа, которые могут использоваться поставщиками публичных облачных услуг для анализа потребностей и тенденций рынка, а также для поиска оптимальных решений, удовлетворяющих максимально возможное количество пользователей с минимальным набором реализованных функций. Этот новый подход дополняет существующие методы выявления требований, поскольку представляет собой специализированный облачный метод, который работает с уже существующими данными, не требуя активного участия потребителей и специалистов по требованиям.

Ключевые слова: облачные сервисы, анализ данных, выявление требований, решетка Галуа

§

1 ВВЕДЕНИЕ

ОБЛАЧНЫЕ вычисления в целом рассматриваются как успешные и Эта парадигма перспективна благодаря своей способности

эффективно адаптироваться к изменениям в бизнесе, гибко масштабируя программные и аппаратные ресурсы. Поэтому в последние годы она вызывает большой интерес как со стороны исследователей, так и со стороны промышленности, и до сих пор ей удалось сохранить обещание обеспечить как технические, так и экономические преимущества [1].

В результате количество доступных публичных облачных сервисов постоянно растет, и ожидается, что этот рост продолжится и в будущем [2]. Хотя это можно рассматривать как преимущество для потребителей облачных услуг, которые имеют широкий выбор предложений, это явление также может привести к парадоксу выбора [3], когда пользователи не знают, какие сервисы лучше всего соответствуют их потребностям. Чтобы решить эту проблему, исследователи [4] и отраслевые специалисты признают, что необходимо разрабатывать поисковые системы или платформы, предназначенные для агрегации и отображения предложений облачных сервисов от различных поставщиков. Они будут действовать как торговые площадки [5], [6], предоставляя расширенные возможности поиска, которые позволят (потенциальным) потребителям облачных услуг вводить и уточнять свои потребности в соответствии с различными критериями. Затем алгоритм сопоставления будет определять, какие существующие публичные облачные сервисы соответствуют запрашиваемым функциям. Примером в этом направлении является Intel Cloud Finder [7], который сопоставляет ИТ-требования с существующими облачными сервисами тех поставщиков, которые зарегистрировались на платформе Intel и опубликовали свое предложение.

Другим следствием быстрого роста популярности облачных сервисов

стало появление множества смежных направлений исследований, от моделей межоблачной архитектуры до производительности и виртуализации облаков. В этом контексте основное внимание уделялось созданию более качественных сервисов с технологической точки зрения, в то время как человеческий фактор в значительной степени игнорировался. Несмотря на наличие примеров исследований в области внедрения облачных технологий [8], [9], вопрос выявления и удовлетворения реальных потребностей потребителей облачных сервисов до сих пор не получил должного внимания.

Более того, существующие методы сбора и выявления требований не подходят для облачной среды, а специализированные методы выявления требований для облачных сервисов отсутствуют [10]. В данной статье мы представляем новый подход к решению существующей проблемы инженерии требований (ИТ): отсутствия специализированных методов выявления требований для облачных сервисов. Мы концентрируемся на выявлении (потенциальных) потребностей потребителей облачных сервисов путем моделирования и анализа данных, собранных из расширенных поисковых запросов потребителей на торговых площадках облачных сервисов.

В следующих разделах мы сначала объясним, что такое выявление требований и почему оно важно. Затем мы рассмотрим проблемы облачных технологий, препятствующие использованию существующих методов выявления требований, и, наконец, изложим основные выводы данной статьи.

1.1 Выявление требований. Выявление

требований обычно рассматривается как первый этап процесса разработки требований [11]. По мнению Соммервилля и Котоньи, выявление требований относится к действиям, предпринимаемым для выявления требований к системе, которую предстоит создать, или к проблеме, которую предстоит решить [12]. Кроме того, Ван Ламсверде также включает выявление заинтересованных сторон в этап выявления требований [13]. Как правило,

Авторы — сотрудники кафедры информатики Цюрихского университета, Цюрих, Швейцария.
Электронная почта: {koitz, glinz}@ifi.uzh.ch.

Рукопись получена 29 января 2015 г.; пересмотрена 19 июня 2015 г.; принята 2 августа. 2015. Дата публикации 11 августа 2015 г.; дата текущей версии 5 октября 2018 г.
Информацию о получении перепечаток этой статьи можно получить, отправив электронное письмо по адресу: reprints@ieee.org, указав ниже цифровой идентификатор объекта.
Цифровой идентификатор объекта № 10.1109/TSC.2015.2466538

Выявление требований подразумевает поиск, сбор и консолидацию требований и рассматривается как необходимый шаг на пути к созданию успешных решений.

Нусейбех и др. подчеркивают, что требования не находятся где-то в ожидании сбора, а необходимы методы выявления для изучения и понимания потребностей пользователей [14]. Например, традиционные методы (например, анкетирование, интервью, анализ существующей документации), методы группового выявления требований (например, мозговой штурм, фокус-группы, семинары RAD/JAD), прототипирование, методы на основе моделей (например, сценарии, KAOS, i*), когнитивные методы (например, анализ протоколов, лестничный анализ, сортировка карточек) и контекстные методы были разработаны для выявления требований и успешно использовались в традиционных условиях на протяжении десятилетий.

Аналогичным образом, в облачной сфере необходимо определить требования потребителей, чтобы определить, какими характеристиками должны обладать будущие облачные сервисы, чтобы удовлетворять их потребности и избегать сбоев. Однако облачная парадигма создаёт ряд проблем, которые не позволяют использовать существующие методы выявления требований, как поясняется в следующем разделе.

1.2 Проблемы облачных

вычислений В то время как существующие методы выявления требований доказали свою эффективность при определении потребностей заинтересованных сторон в традиционных контекстах, т. е. там, где заинтересованные стороны легко идентифицировать и физически доступны, большинство этих методов серьезно осложняются специфическими особенностями облачной среды. Например, учитывая, что облачные сервисы можно легко продавать и настраивать онлайн, потребители, как правило, географически распределены, часто по всему миру. Это приводит к отсутствию локальных рынков, то есть поставщики облачных услуг не всегда хорошо понимают международные рынки, на которых они работают. Это отличается от традиционной модели поставки, когда контракты заключаются с местными физическими поставщиками, которые затем продают им программные или аппаратные решения, способствуя таким образом расширению бизнеса на известный локальный рынок.

Более того, потребители облачных услуг могут быть очень многочисленными и разнородными, с разными профилями и опытом, а также предъявлять совершенно разные требования, которые сложно удовлетворить по отдельности. В таких условиях существующие методы выявления потребностей неприменимы, поскольку невозможно определить заинтересованных лиц, чтобы специалисты по требованиям могли взаимодействовать с ними напрямую [15].

Ещё одной проблемой являются частые запросы на изменение, поступающие от потребителей облачных услуг, особенно от бизнеса, и их изменчивые требования. С одной стороны, такие потребители облачных услуг — это в основном современные компании, которым необходимо быстрое выполнение своих запросов. С другой стороны, чтобы выдержать конкуренцию, поставщикам необходимо сначала знать, в идеале прогнозировать, а затем эффективно удовлетворять эти запросы, чтобы не потерять клиентов. В этом контексте они не могут позволить себе применять методы сбора данных, требующие длительного ожидания сбора требований или длительной обработки и анализа, и именно здесь большинство существующих методов неэффективны.

И последнее, но не менее важное: облачные технологии всё ещё молоды по сравнению с традиционной моделью доставки. Поэтому для решения выявленных проблем мы усовершенствовали предварительный алгоритм, представленный в [18], для удовлетворения потребностей потребителей, которые ещё не разработаны, повысив его производительность с помощью метода обработки, который вычисляет частоту запросов Цумаки и Тамаи [16] для классификации существующих запросов на поиск. Эта функция включена в методы выявления потребностей по двум критериям. Во-первых, мы разработали инструмент StakeCloud.

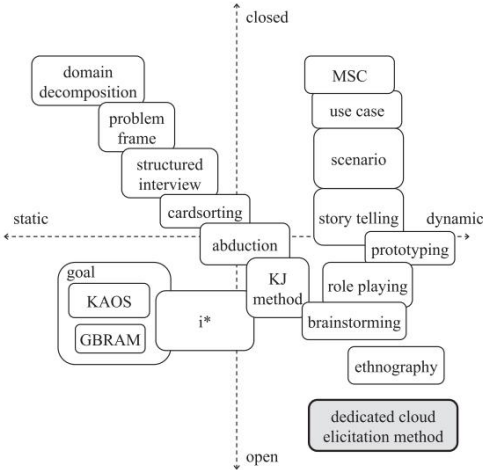


Рис. 1. Методы выявления требований, адаптированные на основе [16].

В зависимости от способа сбора требований, они могут собираться и сортироваться статически или динамически. Во-вторых, в зависимости от свойств целевого пространства, которое необходимо проанализировать, оно может быть закрытым или открытым. Используя эту классификацию, в связи с быстрым и динамичным темпом развития облака, поставщики услуг должны выявлять требования динамически, в идеале непрерывно. Поскольку потребности потребителей могут быстро меняться, и это часто непредсказуемо, пространство является открытым. Как видно на рис. 1, такие методы, как мозговой штурм, ролевые игры или этнография, являются примерами динамических методов, соответствующих этим характеристикам. Однако эти методы обязательно требуют физического и одновременного присутствия заинтересованных сторон в одном географическом пространстве, что несовместимо с облачной парадигмой. Следовательно, согласно существующим работам в этой области и на основе нашего предыдущего исследования [17], очевидно, что существует потребность в специализированных методах выявления требований в облаке, которые помогут облачным компаниям понять своих потребителей и предоставить им услуги, действительно отвечающие их потребностям. Такой специализированный метод будет вписываться в структуру, предложенную Цумаки и Тамаи в нижнем правом углу (динамический процесс выявления и открытое пространство), как показано на рис. 1, и должен отвечать следующим требованиям [17]: R1: подходит для широкой и неоднородной аудитории; R2: занимает меньше времени, чем традиционные методы выявления; R3: обеспечивает возможность автоматизированного выявления; R4: может применяться удаленно; R5: способен обрабатывать изменчивые требования.

1.3 Основные вклады

Эта работа является продолжением статьи, опубликованной на 10-м Всемирном конгрессе IEEE по услугам 2014 года [18]. В ней мы представили предварительную идею построения нечётких решёток Галуа для поддержки облачных провайдеров в работе по выявлению требований. В этом расширении мы представляем три новых вклада:

2. Мы добавляем классификатор сходства, позволяющий гибко кластеризовать похожие запросы. Это приводит к сокращению пространства моделирования и, следовательно, к повышению масштабируемости. Аналогично, эта функциональность включена в прототип.

3. Мы проводим серию экспериментальных оценок, чтобы проверить наш подход к выявлению требований и объяснить, как он соответствует пяти требованиям, указанным выше.

Остальная часть статьи организована следующим образом.

В разделе 2 описывается наш подход, включая идею, определения используемых терминов и разработанные алгоритмы.

В разделе 3 представлена оценка решения и интерпретация результатов. В разделе 4 представлен обзор смежных работ, а в разделе 5 подводятся итоги статьи.

2 ПОДХОД

2.1 Определения

В этом разделе мы вводим термины, используемые при описании нашего подхода, а также соответствующие математические определения.

Определение 1. Частично упорядоченное множество (или частично упорядоченное множество) — это множество, взятое вместе с частичным порядком на нём. Формально частично упорядоченное множество определяется как упорядоченная пара $A \propto \delta$; $\propto$, где X называется основным множеством A , а $\propto$ — частичным порядком A .

Определение 2. Для любого подмножества A_0 частично упорядоченного множества A члены семейств $\text{lb } A_0 \text{ в } \mathcal{A}$ и $\text{ub } A_0 \text{ в } \mathcal{A}$ называются нижней и верхней границами A_0 в A соответственно.

Определение 3. Члены семейств $\inf_{b \in A_0} b$ и $\sup_{b \in A_0} b$ называются инфимой и супремой A_0 в A соответственно.

Другими словами, супремум A_0 — это наименьший элемент A , который больше или равен каждому элементу A_0 . Он уникален и может принадлежать или не принадлежать A_0 . Нижняя грань A_0 определяется аналогично.

Определение 4. Посет A называется решеткой Галуа тогда и только тогда, когда для любого подмножества A_0 множества A существуют наименьшая верхняя граница $\sup 2 A$ (супремум A_0) и наименьшая нижняя граница $\inf 2 A$ (инфинимум A_0).

Соотношения Галуа берут начало в теории Галуа [19] и относятся к соответствию между двумя частично упорядоченными множествами. Согласно Эрну и др. [20], они определяются следующим образом:

Определение 5. Рассматривая частично упорядоченные множества $P \subseteq H_P$; i и $D \subseteq H_Q$; j , если $P \cap Q = \emptyset$. Вопрос и ответ! $P \rightarrow Q$ — это функции такие, что для всех $p \in P$ и всех $q \in Q$, $p \leq q$ тогда и только тогда, когда $p \leq q$, тогда четверка (P, Q, H_P, H_Q) называется связностью Галуа.

Определение 6. Бинарное отношение $R\delta X; Y \vdash$ — это множество упорядоченных пар $\delta x; y \vdash x \in X; y \in Y$. Для любых заданных элементов $p \in X$ и $q \in Y$ пара $\delta p; q \vdash$ либо является элементом $R\delta X; Y \vdash$, либо нет.

В решетке концепций Галуа элементы, как правило, могут принимать двоичные значения. При моделировании запросов к облачным сервисам некоторые характеристики можно легко представить с помощью только двоичных значений, например, наличие или отсутствие поддержки мобильных устройств (1) или отсутствия поддержки (0). Однако многие характеристики лучше представить с помощью шкал отношений, например, для облачных сервисов хранения данных значения A2: проверка на дубликаты. После сбора данных объем может составлять от 1 Гб до 20 Тб; в этом случае наш инструмент StakeCloud, реализующий данный подход, проверяет двоичные значения, и, следовательно, бинарные связи будет сложно использовать. Входной набор данных на наличие дубликатов, т.е. идентичных запросов. Разрешенное лицензирование использования ограничено: Высшей школой экономики. Загружено 12 июля 2025 г. в 06:55:36 UTC из IEEE Xplore. Действуют ограничения.	A1: Сбор данных. Входные данные, необходимые для реализации нашего подхода, представляют собой данные расширенных поисковых запросов, собранные через рынок облачных сервисов. В данном контексте расширенный поисковый запрос — это запрос, позволяющий пользователям указывать желаемые значения для набора предопределенных характеристик.
--	--

Следовательно, мы используем расширение теории решетки Галуа на нечеткие бинарные отношения (fBR) [21], так что признаки могут быть представлены не только в номинальной шкале (принимая двоичные значения 0 или 1), но и в шкале отношений.

Определение 7. Нечеткие бинарные отношения R – допускают степень принадлежности к отношению: степень принадлежности δ_r ; qR к R – может быть любым действительным числом из диапазона $1/2$; 1.

Таким образом, нечёткое множество S_i включает степень принадлежности каждого из своих элементов, принимающую значение в диапазоне $[0, 1]$. Множество, степени принадлежности которого ограничены значениями 0 и 1 (чёткое множество), представляет собой особый тип нечёткого множества, поэтому формально корректно смешивать признаки по номинальной шкале с признаками по шкале отношений в одном и том же представлении.

Определение 8. Каждое понятие в иерархии Галуа, представляющее собой набор объектов, имеющих одинаковые значения для определенного набора свойств, называется **формальным понятием**.

В отличие от общей теории формального анализа понятий (FCA) [22], соотношения Галуа учитывают отношения между нечёткими понятиями, представленными на шкалах отношений. Например, 0,2 и 0,5 — это всего лишь два различных значения в FCA, тогда как 0,2 и 0,5 — это два значения, которые можно упорядочить в теории соотношений Галуа, например, $0,2 < 0,5$. Это приводит к понятию нечётких решёток Галуа [21], узлы которых представляют нечёткие понятия, которые, в свою очередь, построены на основе нечёткого бинарного отношения.

2.2 Идея

С появлением торговых площадок облачных услуг, выступающих в качестве посредников между потребителями и поставщиками, генерируются большие объемы данных. (Потенциальные) потребители используют такие платформы для поиска услуг, соответствующих их критериям, тем самым вводя свои потребности и предпочтения в виде расширенных поисковых запросов. Такие данные журналов в настоящее время используются рекомендательными системами [23] для предложения существующих услуг со схожими функциями, но, как правило, это самая высокая степень эксплуатации этих данных. Значительное большинство компаний, которые мы опросили в одном из наших предыдущих исследований [17], упомянули, что они регистрировали данные поиска потребителей и пытались проанализировать их, но большие размеры обычно затрудняли понимание. В большинстве случаев их анализ сводился к определению того, какие функции услуг чаще всего появлялись в расширенных поисковых запросах. Таким образом, мы сталкиваемся с проблемой больших данных: большие наборы данных невозможно легко обрабатывать и визуализировать, что приводит к потере потенциала таких данных.

Наша идея заключается в использовании зарегистрированных расширенных поисковых запросов для облачных сервисов для выявления требований и комбинаций функций, которые в конечном итоге могут привести к разработке новых облачных сервисов и новых классов облачных решений. Наш подход следует процессу, представленному на диаграмме активности UML на рис. 2. Процесс включает 10 действий, как указано ниже.

A1: Сбор данных. Входные данные, необходимые для реализации нашего подхода, представляют собой данные расширенных поисковых запросов, собранные через рынок облачных сервисов. В данном контексте расширенный поисковый запрос — это запрос, позволяющий пользователям указывать желаемые значения для набора предопределенных характеристик.

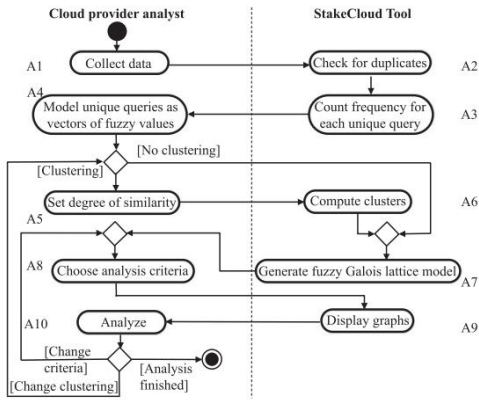


Рис. 2. Диаграмма активности UML процесса нашего подхода [24].

A3: Подсчитайте частоту для каждого уникального запроса. Если дубликатов нет, для каждого запроса частота устанавливается равной 1. Если обнаружены повторяющиеся запросы, инструмент сохраняет только один экземпляр каждого уникального запроса и подсчитывает его частоту, т.е. сколько раз оно встречается в данном наборе данных.

A4: Моделируйте уникальные запросы как векторы нечётких значений. Естественно, запросы расширенного поиска содержат разнородные данные. Данные. Например, значения, связанные с запросом. Входными данными для службы хранения данных публичного облака могут быть: Объем хранилища 300 ГБ, восстановление файлов в течение 30 дней, 98 процентов надёжность, и сервис должен поддерживать шифрование AES для резервного копирования. Использование монотонных функций моделирования, таких как значения преобразуются в нечеткие значения, принимая значения в диапазоне [0, 1]. Например, признак «надёжность» можно смоделировать как монотонную функцию следующим образом: преобразование значения 95 процентов в нечеткое значение 0,5. Здесь диапазон (90,99 процентов) считается самое важное и моделируется соответственно, поскольку большинство услуги не имеют показателей надежности ниже 90 процентов или выше 99 процентов

$$\begin{aligned} & 8 \quad 0; x \ 90\% \\ f\delta \ x\ \> \quad & 0:1 \ \delta \ x \ 90\% ; \ x \ 2 \ \delta \ 90; \ 99\% \\ & : \quad 1; x \ > \ 99\%: \end{aligned}$$

Для характеристик, представленных в номинальном масштабе, таких как «Шифрование AES», $f\delta \ x\ \>$ может принимать значение 1, если функция Доступно, иначе 0. Остальные функции работают аналогично. Очевидно, существует множество способов, которыми $f\delta \ x\ \>$ можно использовать определен; выбор должен только гарантировать, что он будет монотонным преобразование, которое приводит к значениям в диапазоне [0, 1], и поддерживает шкалу отношений для нечётких значений. Таблица 1 показывает список из 10 различных функций облачного сервиса хранения данных, а также с возможными функциями моделирования. Таким образом, передовой Запрос на облачный сервис можно определить как вектор нечетких значения $S_i \ \> f\delta \ x\ \> j \ \> 1; n$; где f представляет особенности облачного сервиса, а n — количество функций присвоено общему типу сервиса S в поисковой платформе. Согласно рис. 2, определение функций моделирования выполняется поставщиком облачных услуг. Однако

наш инструмент дополнен набором predetermined монотонных функции для общих функций облачных сервисов, которые могут быть используется поставщиком облачных услуг для (полу)автоматизации этой задачи. Кроме того, инструмент имеет механизм проверки, который обеспечивает функции, определенные поставщиком, охватывают диапазон [0, 1] Монотонно. Текущий инструмент не поддерживает моделирование. Разрешенное лицензированное использование ограничено: Высшей школой экономики. Загружено 12 июля 2025 г. в 06:55:36 UTC из IEEE Xplore. Действуют ограничения.

ТАБЛИЦА 1			
Характеристики обслуживания (N = Номинальная шкала: {0,1}, R = Шкала отношений: [0,1])			
Нет.	Особенность	Шкала	Функция моделирования $f\delta \ x\ \>$
ϕ_1	Частный пользователь N		0: Н/Д; 1: доступно
ϕ_2	Бизнес-пользователь N		0: Н/Д; 1: доступно
ϕ_3	Хранилище	P	103x; $x < 103$ ГБ; 1; $x \ 103$ ГБ
ϕ_4	Мобильная поддержка N		0: Н/Д; 1: доступно
ϕ_5	Восстановление файлов	P	102x; $x < 102$ дней; 1; $x \ 102$ дня
ϕ_6	Надёжность	P	0; $x \ 90\%$; 1; $x > 99\%$; 0:1 $\delta \ x \ 90\%$; $x \ 2 \ \delta \ 90; 99\%$
ϕ_7	AES-шифрование N		0: Н/Д; 1: доступно
ϕ_8	SSL-шифрование N		0: Н/Д; 1: доступно
ϕ_9	Макс. размер файла	P	0; $x < 0:1$ ГБ; 1; $x \ 10$ ГБ; 0:1 x ; $x \ 2 \ \> 10; 10\%$ ГБ
ϕ_{10}	Время безотказной работы	P	0; $x \ 90\%$; 1; $x > 99\%$; 0:1 $\delta \ x \ 90\%$; $x \ 2 \ \delta \ 90; 99\%$

запросы на исключение, но мы планируем интегрировать это в наш следующий Выпуск. Нежелательные особенности можно отметить в нашем наборе данных флагом, а затем распространить на решётки.

A5: Установите степень сходства. Представитель поставщика облачных услуг может объединить или кластеризовать входящие запросы, выбрав Степень сходства по шкале от 0 до 100 процентов. Степень сходства – это целое число в диапазоне [0, 100] и Сходство 100 процентов означает, что все входные запросы считается на 100% схожим, что приводит к образованию единого кластера, включающего все эти объекты. И наоборот, сходство 0 процентов означает, что все входные запросы различны, таким образом, что число кластеров становится равным числу Запросы, каждый кластер содержит один запрос. При условии, что что степень сходства выше 0, то связывание действие сгруппирует похожие запросы и определит репрезентативный вектор нечетких значений для каждого такого пучка (подробнее об этом (это в разделе 2.3.3). Пакетирование рассчитывается на основе Евклидовы расстояния между векторами признаков, составляющих входные запросы, и супремумом всех Запросы. Поэтому вектор считается похожим на другой. вектор в степени, например, 20 процентов, когда среди всех для других векторов он находится в пределах ближайшего 20-процентного расстояния. Когда входные наборы данных большие, это действие может быть необходимо для более легкой визуализации выходных данных и достичь улучшенной производительности. По этой причине мы позволяем Пользователь может задать степень сходства ещё до создания решётки. Однако пользователь может сгенерировать решётку и напрямую, без объединения.

A6: Вычислить пакеты. При условии, что пользователь выбрал степень сходства отличается от значения по умолчанию (0 процентов), инструмент вычисляет соответствующие пакеты и представитель (как векторы нечетких значений) для каждого. Алгоритм, используемый для этого шага, подробно описан в разделе 2.3.3.

A7: Сгенерировать модель нечёткой решётки Галуа на основе входных данных. набор данных, смоделированный как векторы нечетких значений и/или вычисленные пакеты, инструмент генерирует соответствующие нечеткие Решётка Галуа (пример см. на рис. 7). Решётки графически представляются как ациклические ориентированные графы, имеющие ровно один узел источника (без входящих ребер) и один узел стока узел (без исходящих ребер). В нашем прикладном случае нечёткого Решетки Галуа, узлы на первом уровне иерархии

соответствуют векторам характеристик, предоставленным потребителями облачных данных (например, (1), (2)). Как поясняется в разделе 2.1, самый верхний элемент является супремумом или верхней границей для всех Элементы решетки. Это единственный узел решетки, удовлетворяющий все входные запросы полностью. Однако, реализация такого обслуживания на практике чаще всего либо очень дорогое, либо невозможно. Поэтому мы рекомендуем облачным провайдерам анализировать варианты инфимы, представленные всеми остальными узлами решетки, с точки зрения второй уровень иерархии вниз (например, (1,4), (2,7), (2,4,7), (1,2,3,4)). Узлы Infima – это сервисные предложения, которые удовлетворяют входные запросы, но лишь в ограниченной степени. Таким образом, поставщики могут идти на компромиссы, чтобы удовлетворить большие группы потребителей с минимальным набором требований Реализовано, чтобы достичь оптимального решения. В этом В этом отношении важно отметить, что все элементы infima представляют собой вновь созданные классы услуг, которые могут быть кандидатами на реализацию. Поэтому, наш подход выходит за рамки простой статистики, поскольку Элементы инфимы представляют собой новые комбинации значений признаков, которые не могут быть легко выведены путем подсчета частот в исходных наборах данных или с использованием стандартных статистических методов. Более того, если полученную решетку разбить на подрешетки для более тщательного анализа, те же свойства связанные с инфимой и супремой элементы сохраняются для подмоделей: любая подрешетка имеет по крайней мере супремум и инфимум узел, соответственно, так что в любом В таком подмножестве узлов будет по крайней мере один кандидат на обслуживание, который полностью удовлетворяет узлам в первой иерархии. уровень и по крайней мере узел, который удовлетворяет этим требованиям в ограниченном объеме

Степень, которую можно рассчитать нашим методом. Алгоритм, используемый для построения этих моделей, описан в Раздел 2.3.2.

A8: Выберите критерии анализа. После того, как решетка будет готова, обращается (включая пакеты или нет), пользователь облачного провайдера В рамках нашего подхода мы можем выбрать предпочтительные критерии для анализа данных. Эти критерии выбираются и реализуются таким образом, чтобы они поддерживали сервис. поставщик в его/ее процессе понимания и рассуждения, решать, какие виды услуг следует предоставлять для удовлетворения потребностей потребителей. Например, он(а) может выбрать сравните уровень удовлетворенности отдельными функциями для набор классов услуг (узлов графа), которые он выбрал в решетку, или он(а) может изучать, в какой степени конкретные запросы могут быть удовлетворены новыми классами услуг, созданными нашими Метод. Критерии анализа подробно описаны в разделе 3.5.2.

A9: Отображение графиков. В зависимости от выбранных критериев отображаются соответствующие графики. Графическое представление может состоять из точек или функций в декартовой системе координат. система координат (например, как показано на рис. 7).

A10: Анализ. Представитель облачной компании может использовать решетчатая модель и графические представления от A9 до проводить тщательный анализ запросов потребителей. Более того, он(а) может(ла) в любой момент изменить критерии анализа и создавать новые графики или изменять степень сходства используется для пакетирования.

2.3 Алгоритмы

Действия, составляющие процесс нашего подхода представленные в разделе 2.2, реализуются следующим образом алгоритмы. Разрешенное лицензированное использование ограничено: Высшей школой экономики. Загружено 12 июля 2025 г. в 06:55:36 UTC из IEEE Xplore. Действуют ограничения.

2.3.1 Частотомер

Для поддержки мероприятий A2 и A3 мы используем Алгоритм 1, показанный в псевдокоде ниже, вычисляет частота для каждого запроса входного файла.

```
Алгоритм 1. calculateFrequency(inputFile)

1: d ← dictionary, fuzzyVectorP
2: foreach запрос q 2 inputFile do
3:     если q = 2 d:keys, p тогда
4:         d[q] ← 1
5:     иначе
6:         d[q] ← d[q] + 1
7:     конец, если
8:     конец для
```

В качестве абстрактного типа данных мы используем словарь (или ассоциативный массив), состоящий из пар «ключ; значение». В нашем случае Значения, содержащиеся в словаре, представляют собой векторы нечетких значений (fuzzyVector), и каждый ключ встречается только один раз. каждый запрос q исходного входного файла, мы проверяем, является ли он Часть набора ключей или нет. Если она найдена, мы увеличиваем её счётчик частоты; если нет, частота устанавливается равной 1. Выход состоит из набора данных, созданного исключительно из уникальных запросов и соответствующие вычисленные частоты. Этот алгоритм гарантирует, что набор данных, подлежащий дальнейшей обработке, не содержит дубликаты, что важно для общего Производительность и поведение метода. Алгоритм 1 имеет Сложность O(d * Sp), где S — количество запросов на обслуживание.

2.3.2 Генератор решеток

Действие A7 является одним из основных шагов в нашем подходе, поскольку Он занимается созданием нечёткой решётки Галуа, необходимой модели для анализа данных. Алгоритм 2 показывает какие операции необходимо выполнить перед построением графика.

```
Алгоритм 2. generateLattice(fBR[S,F])

1: C ← C0 ∪ {f};
2: Ck ← C ∪ {δ | S k p};
3: C ← C ∪ {S k ∪ f | Ck Sg};
4: foreach i 2 C do
5:     foreach j 2 i do
6:         foreach f 2 ∪1 :: F do
7:             infFeature[f] ← min(len δ | p r ∪1 j ∪ f);
8:             C0 ← C0 ∪ { infFeature[f] };
9:             конец для
10:        конец для
11:    конец для
12:    исключить дубликаты (C0)
13: drawGraph(C0)
```

Входные данные алгоритма 2 представлены нечетким бинарное отношение, представляющее собой матрицу с двумя измерениями: S сервисные запросы, которые существуют в наборе данных с уникальными запросами, и F-функции, которые являются предопределенными функциями для тип анализируемой услуги, например, облачное хранилище данных, как показано в Таблице 1. Во-первых, все элементы множества мощности Pδ Sp множество S поисковых запросов генерируется как комбинации (строка 2). Это наборы векторов с нечёткими значениями, где S запросов выполняются по k за раз без повторов. Для например, если S ∪5 и k ∪3, C ∪5 ∪f ∪s1; c2; c3; ∪x1; c2; c4; 5 ∪f ∪s1; c2; c3; ∪x1; c2; c4; 5 ∪f ∪s1; c2; c3; ∪x1; c2; c4;

$s_1; s_2; s_5; s_1; s_3; s_4; s_1; s_3; s_5; s_1; s_4; s_5; s_2; s_3; s_4; s_2; s_3; s_5; s_4; s_5;$ где $s_i; i \in \{1, 2, 3, 4, 5\}$ — уникальные запросы на обслуживание. Во-вторых, мы добавляем все вычисленные C_k к C (строка 3), что образует большой набор, включающий все возможные комбинации запросов на обслуживание на основе исходного входного файла.

В-третьих, мы вычисляем нечеткие понятия (строки 4–11), получая $2S_{FC}$. Согласно теории связей Галуа, FC , принадлежащий подмножеству S_0 , вычисляется путем взятия минимумов всех значений признаков запросов, содержащихся в S_0 (строка 7). В случае, если C проявляет особые свойства, они рассматриваются на этом этапе. В качестве типичного примера предположим, что мы обнаруживаем небольшое расстояние между двумя строками нечеткого бинарного отношения. Поскольку этот метод основан на вычислении минимальных значений, обнаружение поискового запроса, являющегося минимумом другого, приведет к возможностям сокращения в итоговой решетке, т.е. некоторые узлы не должны быть представлены из-за избыточности.

Естественно, после расчёта всех FC в C_0 могут оказаться дубликаты. Чтобы не привлекать одни и те же классы услуг дважды, мы исключаем дубликаты из C_0 . Таким образом, мы сохраняем только одну запись для каждого нечеткого вектора, сгенерированного при расчете значений признаков (строка 7). Последний шаг алгоритма 2 заключается в построении решётки (графа). При этом алгоритм учитывает иерархические свойства решётки. Например, наш подход будет строить направленные рёбра $FC_4 \rightarrow FC_5$ и $FC_5 \rightarrow FC_4$ от сервисных запросов s_4 и s_5 , представленных FC_4 и FC_5 соответственно, к узлу решётки $FC_{4,5}$, который представляет собой формальное понятие, сгенерированное на основе запросов s_4 и s_5 . Остальные рёбра строятся аналогичным образом, например, $FC_{4,5} \rightarrow FC_{2,4,5}$, $FC_{2,4,5} \rightarrow FC_{1,2,4,5}$, так что индексы узла супремума всегда представляют собой подмножество индексов узла инфимума.

Операция вычисления минимальных значений в строке 7 имеет сложность $O(2^S)$, что в сочетании со сложностью итерации по комбинациям $O(2^{S_1})$ и признакам $O(F_P)$ приводит к общей сложности алгоритма $O(2^S \cdot F_{22}^2 S^P)$, следовательно, $O(2^S)$. Экспоненциальный характер размера S естественен при вычислении решёток Галуа.

2.3.3 Классификатор сходства Третий

алгоритм используется для реализации действия A_6 (см. рис. 2), где инструмент вычисляет кластеры схожих запросов на основе степени сходства sim , указанной пользователем в действии A_5 .

Алгоритм 3 принимает в качестве входных данных степень сходства sim и множество уникальных запросов ($uniqueQ$). Последнее является выходом операции A_4 и представляет собой двумерную матрицу с S запросами и F признаками сервиса. Алгоритм возвращает $clustersList$, который представляет собой список, содержащий все вычисленные кластеры. Изначально он инициализируется пустым множеством. Наш алгоритм связывания использует евклидовы расстояния между векторами для поиска кластеров, аналогично другим алгоритмам кластеризации из области интеллектуального анализа данных, таким как k-средние. Тем не менее, мы не могли использовать существующую известную реализацию, такую как алгоритм Ллойда, поскольку не можем предоставить количество ожидаемых кластеров (значение k в k-средних), но заинтересованы в получении результирующих кластеров независимо от их количества. В этом отношении наш алгоритм принадлежит к Разрешенному лицензированию использованию, ограниченному: Высшей школой экономики. Загружено 12 июля 2025 г. в 06:55:36 UTC из IEEE Xplore. Имеются ограничения.

Иерархическая кластеризация, позволяющая избежать проблемы предварительного определения количества сгенерированных кластеров. Более того, она имеет детерминированное поведение, всегда выдавая одни и те же результаты для одних и тех же входных данных. Поэтому вместо выбора случайных запросов в качестве якорей для кластеров мы используем один единственный якорь: супремум входного набора данных sup .

Алгоритм 3. calculate Similarity(sim,uniqueQ[S,F])

```
1: clustersList  $\leftarrow \emptyset$ ; 2: если
sim  $\geq 0$  тогда
3: вернуть uniqueQ; F 4: иначе

5: sup  $\leftarrow \sup uniqueQ$  6: uniqueQ
 $\leftarrow uniqueQ \setminus \{sup\}$ ; F 7:
foreach query q  $\in uniqueQ$  do
8:   dist  $\leftarrow \text{distance}(sup, q)$ 
9:   while dist  $\leq \text{BreakingPoint}$ 
10:    for q'  $\in uniqueQ$ 
11:     cluster  $\leftarrow \{q, q'\}$ 
12:     maxDist  $\leftarrow \max(\text{distance}(sup, q), \text{distance}(sup, q'))$ 
13:     if maxDist  $\leq \text{BreakingPoint}$ 
14:      cluster  $\leftarrow \text{cluster} \cup \{sup\}$ 
15:   if dist  $\leq \text{BreakingPoint}$  then
16:    cluster  $\leftarrow \{q\}$ 
17:   end if
18: end for
19: uniqueQ  $\leftarrow uniqueQ \setminus \text{cluster}$ 
20: clustersList  $\leftarrow \text{clustersList} \cup \text{cluster}$ 
21: end while
22: return clustersList

23: конец, если
```

Если заданная пользователем степень сходства sim равна 0, алгоритм возвращает список исходных уникальных запросов (строки 2–3) — список кластеров для $sim = 0$ совпадает с исходным набором данных, когда все входные запросы уникальны. В противном случае, если значение sim больше 0, мы вычисляем максимальное евклидово расстояние между супремумом sup и каждым запросом q в нашем наборе (строки 7–8), а также точку пересечения для кластера (строки 11–13). $BreakingPoint$ определяет нижнюю границу кластера, т. е. наименьшее евклидово расстояние до супремума, в пределах которого должен находиться поисковый запрос, чтобы претендовать на членство в кластере. Все запросы, расстояния которых до супремума меньше значения $breakingPoint$ для данного sim , считаются похожими для указанного значения sim . Каждый элемент набора попадает в кластер, пока расстояние между ним и супремумом меньше точки пересечения (строки 14–18). Затем набор данных обновляется (строка 19), чтобы включить в него только те элементы, которые не стали членами кластеров или были исключены как отдельные некластеризованные элементы, и описанные выше шаги повторяются до тех пор, пока набор данных не станет пустым.

Наконец, кластеры, сформированные в каждой итерации, добавляются в $clustersList$ (строка 20).

Для вычисления репрезентативного объекта каждого кластера (вектора нечётких значений) мы используем концепцию центроида или геометрического центра. Он рассчитывается как среднее арифметическое положение всех точек в n -мерном пространстве, где n — количество признаков для указанного облачного сервиса. Мы не выбрали концепцию медоида в качестве репрезентативного объекта для каждого кластера, как это обычно делается в данных.

майнинг, поскольку медалиды всегда являются элементами набора данных, а в нашем случае это не было обязательным требованием. Здесь важнее добиться минимального различия между репрезентативным образцом и всеми элементами кластера, чтобы новая решетка, сгенерированная после объединения, хорошо иллюстрировала исходный набор данных без значительных потерь информации.

Если запросы, включенные в кластер, имеют частоты выше 1, они также учитываются, поскольку мы вычисляем взвешенный центроид, т.е. каждый запрос представлен в кластере пропорционально его частоте.

Учитывая, что мы сначала проверяем, равно ли sim нулю, сложность алгоритма 3 в лучшем случае составляет $O(n \log n)$. В противном случае, поскольку вычисление maxDist выполняется за $O(n^2)$, общая сложность составит $O(n^3)$, где n — количество элементов uniqueQ , а n_0 — количество элементов uniqueQ_0 . Сложность в худшем случае стремится к $O(n^2)$, когда n_0 стремится к n .

3 ОЦЕНКА

3.1 Цель и показатели

Чтобы оценить наш подход, мы оцениваем его соответствие пяти требованиям, представленным в разделе 1.2 (R1–R5). Эти требования были выявлены в ходе исследований в области методов выявления потребностей в выделенном облаке и в ходе нашего предыдущего исследования с участием 19 компаний-поставщиков облачных услуг [17]. Поэтому мы предполагаем, что наличие подхода, отвечающего этим требованиям, поможет поставщикам облачных услуг лучше понимать и удовлетворять потребности своих клиентов. Мы определяем цель нашей оценки следующим образом:

Проанализируем наш специализированный подход к выявлению требований с целью оценки того, в какой степени он удовлетворяет требованиям поставщиков облачных услуг к новому методу выявления требований (R1–R5) с точки зрения компаний — поставщиков облачных услуг в контексте анализа расширенных поисковых запросов для облачных сервисов с целью вывода новых требований.

Поскольку R2 отвечает за эффективность использования времени, основной метрикой, используемой для выполнения этого требования, является время. Мы рассчитываем время, необходимое для генерации узлов решетки и моделей, а также время, необходимое для поиска похожих запросов и генерации кластеров. Что касается автоматизации (R3), мы показываем, какие выходные данные могут быть автоматически получены с помощью нашего подхода, и объясняем, в какой степени представленный метод более автоматизирован, чем существующие методы выявления требований. Кроме того, мы используем стандартное отклонение в качестве метрики неоднородности (R1) запросов в наборе данных. Что касается удаленного приложения (R4) и изменчивых требований (R5), мы описываем, как наш подход может быть применен, как его выходные данные могут быть проанализированы удаленно, а также как можно отслеживать изменчивые требования и делать будущие прогнозы.

3.2 Метод

В данной работе мы фокусируемся на внутренней, а не на внешней оценке. Это означает, что мы оцениваем эффективность нашего подхода и показываем, насколько он соответствует выявленным требованиям (R1–R5), но не проводим внешнюю оценку по сравнению с другими существующими методами. Основная причина этого заключается в том, что такая оценка практически невозможна.

ТАБЛИЦА 2					
Идеи менеджеров по продукту по использованию образца набора данных					
Активность	PM1	PM2	PM3	PM4	PM5
Анализ частоты запросов	x		x		x
Анализ важности запроса					x
Прогнозировать будущие запросы			x		x
Анализировать запросы на услуги, похожие на их				x	x

Ни один из существующих методов выявления требований, упомянутых в разделе 1.1, не использует в качестве входных данных наборы данных расширенных поисковых запросов, что позволило бы нам сравнить результаты нашего метода с результатами других аналогичных методов. В связи с этим инновационный характер нашего подхода обуславливает отсутствие эталона или исходного значения, которые мы могли бы использовать для внешней оценки, например, с использованием таких метрик, как точность, полнота или количество ошибок. Более того, попытка вручную проанализировать наборы данных для построения собственного исходного значения невозможна, учитывая большие объемы данных.

3.3 Вклад менеджеров по продукту (PM). В наших полуструктурированных интервью, проведенных с поставщиками облачных услуг в период с ноября 2012 года по январь 2013 года, посвященных выявлению требований [17], мы обсуждали возможность использования данных поиска потребителей для определения требований к сервисам. В то время, хотя некоторые компании регистрировали такие данные, ни одна из 19 опрошенных облачных компаний не использовала их для выявления требований. В январе 2015 года мы снова связались с пятью менеджерами по продукту из компаний, расположенных в трех разных европейских странах, с которыми мы ранее беседовали, и спросили их, изменилась ли ситуация за это время.

Все пятеро отметили, что, хотя идея использования анализа данных журналов для лучшего понимания своих (потенциальных) потребителей постоянно обсуждается, никаких конкретных инициатив в этом направлении пока не предпринято. Они подтвердили, что подходы к выявлению требований, которые они использовали во время наших интервью, продолжают применяться и сейчас.

Более того, теперь мы дали пяти менеджерам по продуктам образец набора данных из 500 расширенных поисковых запросов и спросили их, как они будут использовать его, чтобы помочь своим компаниям в процессе принятия решений относительно запуска нового облачного сервиса. Наиболее частые зарегистрированные ответы показаны в Таблице 2. Анализ частоты каждого запроса был наиболее распространенным действием, которое они предпримут (четыре из пяти ответов), рассуждая, что если много людей ищут один и тот же тип сервиса, это признак того, что такой сервис следует запустить, если его не существует. PM2 и PM4 упомянули, что несколько таких наборов данных из разных моментов времени можно использовать для прогнозирования будущих запросов на основе истории развития. PM3 и PM4 также добавили, что было бы интересно проанализировать те запросы, которые похожи на то, что уже предлагают их компании, чтобы определить, какие изменения можно было бы внедрить для улучшения их сервисов и удовлетворения потребителей. Самый продвинутый подход предложила PM5, которая упомянула, что будет использовать метод tf-idf (частота терминов — обратная частота документов) для определения важности запроса в наборе запросов (документ) из коллекции наборов запросов (корпус). Это похоже на анализ отдельного запроса.

Частота, поскольку tf-idf пропорционален частоте каждого запроса, но компенсируется частотой запроса в корпусе. На вопрос, используют ли они такие наборы данных, три менеджера проектов ответили, что их рекомендательные системы — единственные, кто использует такие данные, чтобы предлагать сервисы, похожие на те, которые просматривает пользователь.

Таким образом, хотя пять опрошенных менеджеров проектов слишком малы для достижения статистической значимости, они подтвердили нашу гипотезу об отсутствии методов выявления требований, использующих те же входные данные, что и наш метод. Более того, наш инструмент, реализующий этот подход, поддерживает автоматизацию действий, предложенных менеджерами проектов, и выходит за их рамки. С одной стороны, эти результаты побуждают нас продолжать работу по разработке представленного подхода. С другой стороны, они подтверждают, что найти истинную основу для внешней оценки практически невозможно. Поэтому мы провели оценку, описанную ниже.

3.4 Экспериментальная установка.

В качестве данных для моделирования мы использовали три различных набора данных, представляющих собой запросы расширенного поиска для облачных сервисов хранения данных. Они содержат 250 запросов¹, 500 запросов² и 1000 запросов³ соответственно. Поскольку получение больших объёмов реальной поисковой информации особенно затруднено из-за конфиденциальности и конфиденциальности, мы самостоятельно сгенерировали необходимые наборы данных.

Однако мы использовали небольшой доступный набор данных, содержащий реальные данные [18], полученный от одной из опрошенных компаний [17] в качестве отправной точки. Более того, мы сгенерировали запросы таким образом, чтобы они определяли облачные сервисы хранения данных с 10 характеристиками, как описано в Таблице 1. Для этого мы следовали ограничениям, накладываемым отдельными характеристиками: некоторые из них представлены в номинальной шкале, тогда как другие представлены в относительной шкале. Таким образом, некоторые характеристики определяются двоичными значениями, такими как «мобильная поддержка», а другие — нечеткими значениями в диапазоне [0, 1], такими как «надежность». Для проверки производительности классификатора сходства мы устанавливаем степень сходства от 20 до 90 процентов.

Все эксперименты, представленные в следующих разделах, проводились на процессоре Intel Core i7 с частотой 1,8 ГГц и памятью DDR3 объёмом 4 ГБ и частотой 1333 МГц.

3.5 Автоматизация и эффективность использования

времени. Мы увидели, что существующие методы выявления требований в облачной среде сталкиваются с проблемой времени. Более того, необходимость одновременного привлечения большого количества заинтересованных сторон в одной географической точке — ещё одна проблема, возникающая в облачных средах, которую можно решить с помощью автоматизации. Чтобы показать, как наш подход решает эти проблемы, мы сначала проанализируем эффективность использования времени (R2) наших алгоритмов, а затем опишем возможности автоматизации (R3).

3.5.1 Эффективность времени (R2)

В отличие от существующих методов сбора информации, наш подход имеет пассивный характер, т.е. потребители не

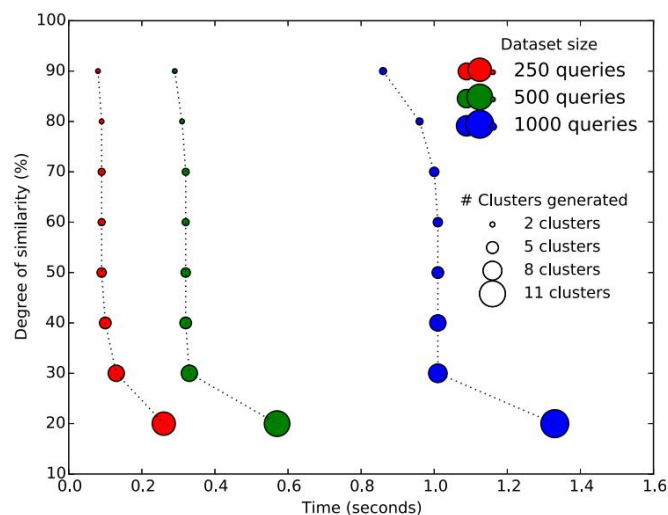


Рис. 3. Временные характеристики классификатора сходства и количество сгенерированных кластеров.

непосредственно и осознанно вовлечены в процесс выявления требований, поскольку требования к новым сервисам определяются на основе их поисковых запросов. Таким образом, время практически не тратится непосредственно на процесс выявления, а тратится на анализ данных. Следовательно, мы рассчитаем время обработки данных.

Классификатор сходства. Сначала мы анализируем поведение классификатора сходства (алгоритм 3). Как описано в разделе 2.3.3, степень сходства (sim) определяется представителем облачного провайдера с использованием прототипа StakeCloud [24], реализующего наш подход. Затем автоматически генерируются группы похожих запросов. Мы измерили время, необходимое для генерации кластеров со степенью сходства от 20 до 90 процентов с шагом 10, для трёх наборов данных по 250, 500 и 1000 запросов соответственно. Мы исключили крайние значения диапазона [0, 100], т.е. [0, 20] и (90, 100], поскольку значения, приближающиеся к 0, не будут генерировать никаких кластеров и просто сохраняют исходный список запросов, а значения, приближающиеся к 90, скорее всего, сгенерируют только один большой кластер, состоящий из всех запросов в наборе данных. Более того, мы измерили, сколько кластеров генерируется для каждой степени сходства для каждого набора данных. Полученные результаты показаны на рис. 3.

Как правило, время, необходимое для обработки данных и формирования групп запросов, линейно растёт с ростом числа запросов в наборе данных: например, при sim = 20 генерация кластеров для набора данных из 250 запросов (показано красным на рис. 3) займёт 0,26 секунды, тогда как для набора данных из 500 запросов (зелёный) — 0,57 секунды, а для 1000 запросов (синий) — 1,33 секунды. Это наблюдение справедливо и для других значений степени сходства. Полученные нами значения времени находятся в том же диапазоне, что и значения известных методов кластеризации, таких как k-средние [25], для наборов данных сопоставимых размеров.

Как и ожидалось, время, необходимое для генерации кластеров, прямо пропорционально количеству сгенерированных кластеров и обратно пропорционально значению sim. На рис. 3, чем больше размер круга, представляющего кластер, тем большее количество кластеров генерируется для данной степени сходства, как показано в легенде. Например, для набора данных с 500 запросами (зелёный) sim = 90 приведёт к двум кластерам, тогда как sim = 20 приведёт к 11.

1. <http://www.ifi.uzh.ch/rerg/people/todoran/Dataset250.pdf> 2. <http://www.ifi.uzh.ch/rerg/people/todoran/Dataset500.pdf> 3. <http://www.ifi.uzh.ch/rerg/people/todoran/Dataset1000.pdf>

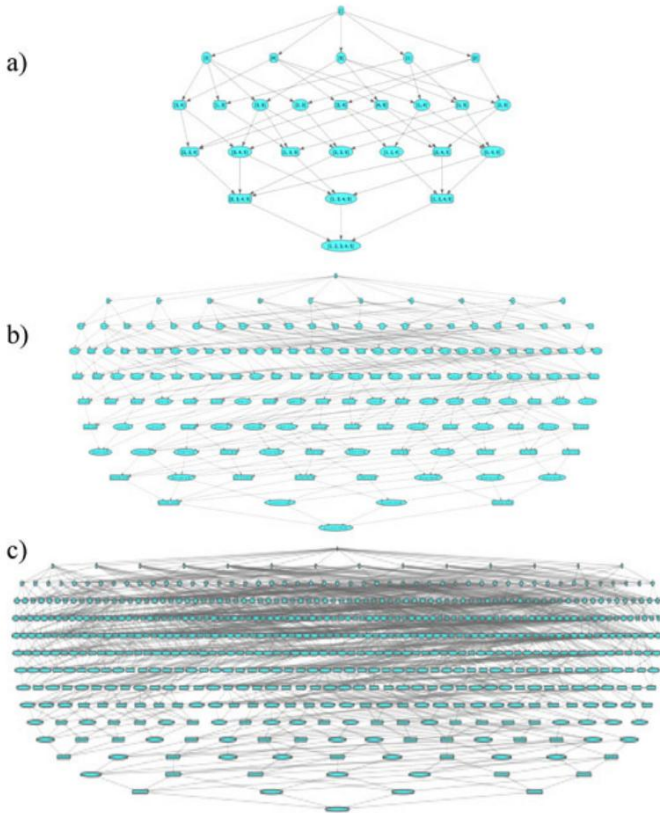


Рис. 4. Нечеткие решетки Галуа для: а) пяти запросов, б) 10 запросов и в) 15 запросов запросы.

кластеров. Можно отметить, что значение $\text{sim} \frac{1}{4}20$ вызывает сравнительно более длительные необходимые периоды времени по сравнению с другими значениями. Это связано с тем, что наш алгоритм всегда сравнивает расстояния между супремумом и каждый запрос к точке критического расчета (строка 15, в Алгоритм 3), и чем меньше значение сим, тем больше таких необходимы вычисления. Тем не менее, на обычной машине время всё ещё поддается контролю, находясь в диапазоне секунд.

Согласно нашим текущим тестам, визуализация решеток Галуа сгенерированный непосредственно из наборов данных с менее чем 15 запросами всё ещё возможно, без необходимости кластеризации, если это специально не требуется. Инструмент позволяет масштабировать и перетаскивать, что облегчает пользователю навигацию в моделях даже когда графики довольно сложные, как на рис. 4б и 4с. Для наборов данных с более чем 15 запросами пользователь может выберите автоматический расчет пакетов и затем сгенерируйте решетку. Таким образом, он может регулировать степень сходства Пока модель не станет легко визуализироваться. Когда некоторые из представленных узлов не являются исходными запросами из набора данных, а представителей пакетов, функция наведения мыши позволяет пользователь может увидеть, из каких составных запросов состоит каждый кластер, и их частоты.

Генерация решётки. Мы вычислили время, необходимое для генерации нечётких решёток Галуа для различных наборов данных, содержащих От пяти до девятнадцати запросов. Мы протестировали наш Алгоритм 2 с наборы данных таких размеров, поскольку они, вероятно, приведут к моделям которые по-прежнему легко визуализировать и анализировать, как объяснялось выше. По этой причине мы обычно рекомендуем объединять изначально аналогичные запросы, если входной набор данных содержит больше чем 15 запросов, так что общий вид данных будет первым

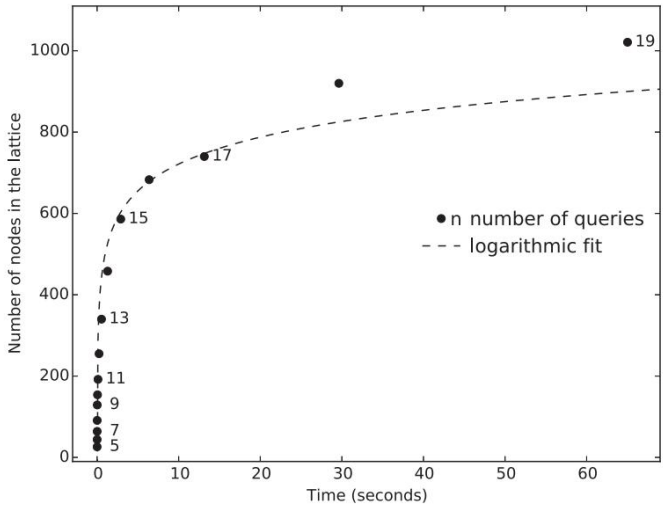


Рис. 5. Время, необходимое для генерации узлов решетки.

Затем пользователь может изучить отдельные пакеты. и пересоздать модель по мере необходимости. Это также гарантирует быстрая генерация решетки, а значит и быстрая обработка данных: например, построение модели для 10 отдельных запросов занимает 0,05 секунд, генерируя в общей сложности 154 узла для представления в решетке.

График на рис. 5 показывает время, необходимое для генерации соответствующие узлы решетки для используемых наборов данных. Метки, отображаемые рядом с точками, представляют собой число обработанных запросов. Наборы данных из пяти-тринадцати запросов создают решётки практически мгновенно, менее чем за 1 секунду. Затем необходимое время увеличивается по экспоненциальной зависимости. Логарифмическая кривая аппроксимации для полученного точек отображается пунктирной линией и имеет следующее определение:

$$y \approx x^p \cdot \log b$$

В то время как метод действительно основан на комбинации В начальных запросах применяется несколько сокращений. Во-первых, Частота каждого запроса рассчитывается, что позволяет отобразить только уникальные запросы. В случаях, когда значения частоты выше 1 обнаружены, они помечены и размножены в новых классах услуг. Во-вторых, когда Вектор нечетких значений, представляющий запрос, является минимальным другого вектора, генерируются многочисленные узлы, которые идентичны; снова, только уникальные новые классы услуг Включены в решетку, что является еще одним сценарием сокращения. В-третьих, запросы к реальным наборам данных не как правило, совершенно различны, и чем больше они похожи, Чем меньше узлов решетки, тем меньше их. Следовательно, несмотря на предполагаемый экспоненциальный характер, наш подход работает лучше, чем 2п. Комбинации, изначально вычисленные Алгоритм 2 касается только меток запросов и Фактические вычисления выполняются только на узлах, которые претендовать на то, чтобы стать частью решетки.

Применяемые сокращения приводят к высоким значениям дельта (D) между числом комбинаций, сгенерированных математически, и фактическим числом узлов, необходимых для построения решетки. Это показано на рис. 6, где представлено расхождение между числом узлов решетки (серый) и их комбинаций (черный) для пяти, 10, 15 и 20 запросов соответственно.

Например, для набора данных из 15 запросов $D = 1; 047; 464$. Разрешенное лицензированное использование ограничено: Высшей школой экономики. Загружено 12 июля 2025 г. в 06:55:36 UTC из IEEE Xplore. Действуют ограничения.

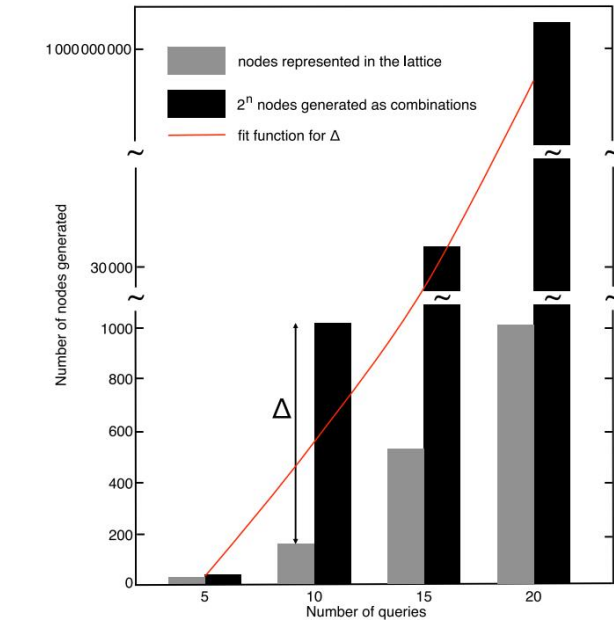


Рис. 6. Узлы решетки и их комбинации.

Экспоненциальная кривая аппроксимации для средних точек значений D имеет вид показан красным цветом и имеет определение:

$$y \approx x^p \quad 1/0:54 \text{ e}0:69x:$$

Экспоненциальный характер этой кривой соответствия демонстрирует что в то время как время, необходимое для создания решеток, растет Этот подход работает быстрее, чем стандартный экспоненциальный, не требуя всех математических комбинаций. Когда входные наборы данных содержат менее 50 уникальных запросов, время вычислений остается в диапазоне часов, например, в течение 25 запросы, без какой-либо предварительной обработки и объединения, необходимое время составляет 3,85 ч. Тем не менее, как только будет произведено объединение При применении время может значительно сократиться, в зависимости от Значение степени сходства. Этот вопрос эффективности

известный в области концептуальных решеток и различных решений были расследованы с целью смягчения последствий. Например, Кумар и Шринивас [26] применяют кластеризацию методом k-средних для редукции решётки, определяя число желаемых кластеров k. Однако, они фокусируются только на кластерах, полученных в результате их анализа, а не ни слова о приросте производительности. В нашем контексте, мы можем сделать вывод, что время, необходимое для данных анализ все еще намного короче, чем у любого другого существующего Методы выявления требований, даже когда их несколько Требуется часы вычислений. Более того, результаты сообщалось, получены на нашем довольно ограниченном рабочем столе Машина. Алгоритм 2 в принципе поддается распараллеливанию, и его запуск на многоядерном кластере должен дать лучшие результаты. Уменьшение времени вычислений пропорционально количеству используемых ядер. Однако это вопрос будущих исследований.

3.5.2 Автоматизированное выявление (R3)

Что касается автоматизации, наши решетки Галуа Подход к облачным сервисам отличается от всех существующих Методы выявления требований при сборе данных Полностью автоматизирован. Потребности собираются у (потенциальных) потребителей пассивным и ненавязчивым образом. Более того, наша технология поддерживается инструментами [24], поэтому большинство Анализ данных автоматизирован. Как показано на рис. 2, Инструмент StakeCloud выполняет множество действий, таких как вычисление групп похожих запросов. В то время как новый Классы услуг генерируются автоматически, при выполнении анализа в A10 поставщики могут вручную провести исследование «что если», чтобы динамически моделировать то, что происходит, когда изменяется только одна или несколько характеристик, как они влияют на общую кластеризацию или увеличивают определенные части решетки, чтобы проанализировать лучшие идеи для новых услуг.

Например, на рис. 7 показан пример сценария в нашем инструменте. Верхняя правая панель содержит загруженный входной набор данных, состоящий из 20 различных запросов. Соответствующая решетка Галуа отображается на главной панели, где самый верхний элемент является супремумом для всей решетки, и первый

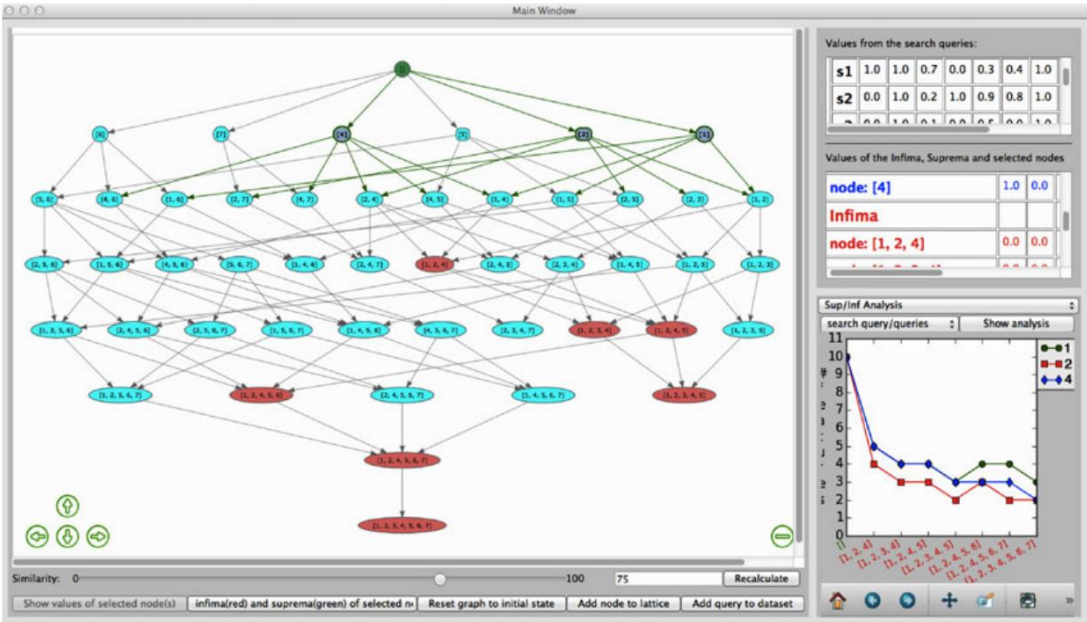


Рис. 7. Скриншот инструмента — пример сценария.

Таким образом, благодаря предоставляемой инструментальной поддержке мы предлагаем наш подход как методику выявления требований, которая позволяет в значительной степени автоматизировать получение требований и предоставляет поставщикам облачных услуг средства для анализа автоматически сгенерированных визуализаций.

В этом разделе мы покажем, как наш подход решает задачи R1, R4 и R5, позволяя выявлять требования глобально распределенной аудитории, чьи потребности изменчивы.

Расширенный поиск, необходимый для нашей методики нечетких решеток Галуа, всегда осуществляется на веб-сайтах торговых площадок. Таким образом, наш подход позволяет любому количеству потребителей практически из любой точки мира вводить свои потребности в услугах полностью асинхронно. Более того, по словам опрошенных поставщиков облачных услуг [17], расширенный поиск услуг — один из наиболее часто используемых методов как частными потребителями, так и компаниями для выбора облачных решений, соответствующих их потребностям.

В нашем случае N — количество запросов, $x_i, i = 1; N$ представляет отдельные запросы, а m — среднее значение запросов в наборе данных. Мы использовали стандартное отклонение для измерения величины дисперсии от среднего значения для запросов в наших трёх наборах данных и получили следующие результаты. Для набора данных, состоящего из 250 отдельных запросов, $s_{250} = 0.324$, для 500 запросов — $s_{500} = 0.322$, а для 1000 запросов — $s_{1000} = 0.319$. Все эти значения округляются до значения 0,32, что указывает на то, что все запросы, использованные в наших тестах, хорошо распределены, следовательно, неоднородны.

Анализ может продолжаться до тех пор, пока поставщик не получит достаточно информации для принятия решения о том, какие услуги стоит запустить, например, путем разделения запроса (2) для анализа запросов, из которых он состоит, создания графиков, которые наглядно показывают, в какой степени запрос (7) может быть удовлетворен заданной нижней границей, создания графиков, которые также показывают степень

в котором характеристики выбранных узлов частично удовлетворены и т. д. Другой пример анализа см. в [18]. Этот анализ проводится исключительно дистанционно, на основе входной набор данных. Учитывая его ненавязчивый характер, этот метод также подходит, когда потребители не могут описать свои требования легко на собеседовании или семинаре, и может таким образом, его можно использовать в качестве дополнения к другим методам выявления.

3.6.3 Нестабильные требования (R5)

Нестабильные требования, т.е. требования, которые меняются в то время как выявляются, анализируются, проверяются и/или внедряются, представляют собой признанную проблему для методов выявления требований. В отличие от существующих методов, где много времени отводится на выполнение требований первого сбора которые затем анализируются, наш подход обеспечивает непрерывный процесс сбора данных. Данные потребителей облачных услуг собираются постоянно, как и расширенные поисковые запросы на торговых площадках, и могут быть мгновенно использованы в качестве входных данных для нашего инструмента, поддерживаемого Подход. Таким образом, можно непрерывно отслеживать изменчивые требования. Более того, благодаря дистанционный характер нашего метода, это делается практически Никаких дополнительных затрат. Эта функция особенно подходит для гибкий характер большинства компаний-поставщиков облачных услуг, которые используют быстрые циклы разработки и имеют довольно короткие время выхода на рынок.

Более того, наш подход можно использовать для мониторинга тенденций. Например, он позволяет рассчитать среднее значение для выбранных характеристики серии наборов данных за определенный период времени. Если это обнаружили, что среднее значение постоянно меняется в течение этого периода времени, это представляет собой намек на то, что эта функция может последовать Эта тенденция сохранится и в будущем. Например, если среднее значение для функция «хранение» увеличивается на 0,5 ГБ каждый квартал для два года, это может означать, что рост должен быть ожидается также в следующем году(ах). И наоборот, если частота двоичной функции, такой как «шифрование AES», уменьшается в течение заданной серии наборов данных, это может быть указание на то, что эти функции могут быть заменены другими, или потребители просто больше не хотят этого.

3.7 Угрозы действительности

Что касается конструктивной валидности, мы старались избежать опасения оценки, гарантируя, что менеджеры по продукту связались с нами, что вся информация анонимна и используется исключительно в исследовательских целях. Мы также смягчили попытки домыслить гипотезу, не предоставляя им никаких подробностей о наших Подход. Внутренняя валидность находится под угрозой из-за того, что мы Мы сами сгенерировали наборы данных. Однако мы построили их, уважая ограничения закодированных функций и Начав с реального набора данных. Таким образом, мы сократили Возможная причинно-следственная связь между лечением и результатом. Тот факт, что наборы данных генерируются самостоятельно, вызывает опасения. внешняя валидность, т.е. обобщение результатов на промышленные В частности, это касается практики. Однако мы попытались построить репрезентативные запросы, как описано выше.

4 СВЯЗАННЫЕ РАБОТЫ

С начала 2000-х годов исследователи заметили, что при проектировании требований также необходимо учитывать распределенные [27] и асинхронные настройки [28], и это в настоящее время расширяется Разрешенное лицензированное использование ограничено: Высшей школой экономики. Загружено 12 июля 2025 г. в 06:55:36 UTC из IEEE Xplore. Действуют ограничения.

в контексте облака. Однако из-за интенсивной совместной работы и больших временных затрат сбор требований в облаке становится затруднительным [27], [29].

Что касается методов выявления требований к выделенному облаку обеспокоены, что в течение этого времени наблюдался определенный прогресс В последние годы. Например, фреймворки, ориентированные на были разработаны соотношения спроса и предложения [30], Сан и др. разработали гибридную нечеткую структуру, которая поможет потребителям облачных услуг выбирать услуги, соответствующие их потребностям. потребности, когда их запросы неопределенны [31], и системы управления требованиями, гарантирующими качество обслуживания (QoS), имеют были разработаны [32]. Кроме того, исследователи изучали методы выявления определенных типов требований, Например, юридические [33] или охранные [34]. Тем не менее, это лишь нишевые рекомендации и отсутствие всеобъемлющего четкого решения существует, решая проблемы выявления требований, специфичных для облака.

Что касается выявления распределенных требований, Ллойд и др. провели исследование [15] эффективности методов выявления распределенных требований инженерия, заключая, что синхронное выявление Подходы, как правило, более эффективны, чем асинхронные. Лим и др. [35] представляют идеи об асинхронных и распределенная идентификация заинтересованных сторон, предполагая, что Ключевые заинтересованные стороны известны, и дополнительные пользователи могут быть идентифицированы на основе знаний предметной области. Однако такие подходы нелегко распространить на контекст облака, поскольку Аудитория услуг чаще всего неизвестна и распределена по всему миру.

Туунанен [36] рассматривает проблему достижения и с участием широкой аудитории конечных пользователей или пользователей, которые не в пределах организационной досягаемости. Он утверждает, что традиционные методы не обеспечивают адекватных решений и представляют Методы, которые потенциально могли бы заполнить этот пробел (например, Easy-WinWin). Однако ни один из этих методов не был успешно использован в больших масштабах для распределённого сбора данных, поэтому Далеко. Более того, исследование EasyWinWin, проведенное Кукрейей, и др. [37] обещают обеспечить поддержку распределенных настроек, но фокусируются только на заинтересованных сторонах в пределах организационной досягаемости.

Еще одна проблема выявления требований в облако – это постоянное изменение потребностей потребителей [38]. В результате были реализованы различные подходы вики, чтобы обеспечить возможность быстрого обновления и выявление требований. Например, Декер и др. разработали решение на основе вики, которое позволяет заинтересованным сторонам участвовать в RE [39], а также пространственное гипертекстовое вики Солиса и Али фокусируется на распределенных командах [40], тогда как Лян и др. [41] и Ломанн и др. [42] используют семантические аннотации вики. Однако методы, основанные на вики, обычно предполагают, что заинтересованные стороны, по крайней мере, могут быть идентифицированы, что не соответствует действительности в облачный контекст.

Исследования в области веб-информационных систем Ян и Тан [43] показывают, что для выявления потребностей относительно интернет-систем также довольно сильно отличаются от традиционных систем (например, из-за более высокой пользовательской разнообразие). Более того, большинство существующих методов выявления требований могут работать только с ограниченным числом [38] Количество потенциальных облачных сервисов Потребители часто могут выходить за рамки традиционных методов может справиться [34], и нет реальных решений, которые бы это исправили

Проблема выявления уже разработана. Рыночные методы [44], которые обычно применяются, когда невозможно рассматривать отдельных потребителей, доказывать быть довольно ограниченным в облаке из-за отсутствия конкретных локализованных рынков.

Работа в области интеллектуального анализа данных, машинного обучения и, в частности, Рекомендательные системы [5] также решают проблему Извлечение ценности из поисковых данных. Например, методы, основанные на поиске и сотрудничестве, могут сделать персонализированные онлайн-рекомендации по продуктам [45] и отзывы пользователей использовался для ранжирования различных продуктов [46]. В последние годы рекомендательные системы [23] (например, основанная на вероятности совместная фильтрация [47]) и кластеризация данных Методы добычи [48] широко использовались для маркетинга для предложения аналогичных товаров в системах электронной коммерции или для сегментации аудитории. Однако, насколько нам известно, подобные методы никогда не были адаптированы и не использовались. для выявления требований в облаке.

Кроме того, методы анализа данных, моделирующие потребности потребителей, не были исследованы для целей облачных вычислений. В то время как в области концептуальных решеток Галуа проводится обширный объем исследований, большинство исследователей сосредоточены исключительно на математическом последствия таких графиков и не применять их в Практический контекст. Есть лишь несколько примеров попыток где решетки используются для идентификации объектов или концепций в заданных наборов данных [49], [50] или примененных к поиску в браузере [51]. Большинство существующих работ, связанных с решетками, фокусируются на кластеризации. возможности с решетками Галуа с чисто теоретической точки зрения перспектива [52].

Подводя итог, можно сказать, что существующие методы выявления, даже при адаптации в большинстве случаев не подходят для облака и могут поддерживать поставщиков облачных услуг только в ограниченной степени их процессы выявления требований. Более того, существующие подходы к интеллектуальному анализу данных и решетчатых моделям не были применялось в контексте приобретения требований до сих пор, чтобы наши знания, оставляя вопрос о выделенном облаке нерешенные методы выявления требований.

5. ЗАКЛЮЧЕНИЕ И ДАЛЬНЕЙШАЯ РАБОТА

В этой статье мы предложили новый подход к выявлению требований для облачных сервисов, который создает нечеткие Решетки Галуа, основанные на расширенном поиске потребителей Запросы. Мы сравнили его с пятью основными поставщиками облачных услуг. требования и показал, как он их решает: он может собирать потребности широкой и разнородной аудитории чьи требования изменчивы, автоматизирует анализ данных и может применяться удаленно, занимая меньше времени, чем традиционные методы выявления. Наш подход лучше всего подходит для ранняя фаза выявления и для мониторинга рынка Тенденции. На смену им могут прийти более глубокие требования. Извлечение информации с помощью дополнительных методов, таких как прототипирование и масштабные онлайн-эксперименты. Хотя наш подход изначально был разработан для облачных контекстов, мы предвидим что его можно успешно применять и в других областях которые демонстрируют схожие свойства и где запросы могут быть собранных аналогичным образом, например, в традиционных веб-сервисах и сервисно-ориентированных системах. Однако мы не До сих пор мы исследовали этот сценарий использования, но он может быть изменен будущая работа.

Ограничением нашего подхода является то, что он предполагает, что потребители предоставляют значения для всех функций, указанных как расширенные. Критерии поиска. В настоящее время мы игнорируем запросы со значениями нулевых значений для всех признаков (выбросов) и присваиваются значения по умолчанию когда значения не назначены. Сейчас мы работаем над реализация оценки максимального правдоподобия, которая использует имеющиеся данные для вычисления максимального правдоподобия Оценки. Более того, мы планируем улучшить временные характеристики и масштабируемость метода, используя MapReduce для вычисления минимальных значений в алгоритме 2. Кроме того, мы в настоящее время работаем над расширением подхода к работе, когда отдельные характеристики имеют вес, указывающий на их важность. Естественно, проблемы принятия связаны с конфиденциальностью данных пользователей. может возникнуть, когда наш метод применяется на практике и Может потребоваться некоторое время, пока поставщики облачных услуг привыкнут рабочий процесс. Поэтому мы планируем применить нашу методику выявления метод в реальных условиях, с облачными провайдерами и свои наборы данных, чтобы обнаружить любые потенциальные проблемы и продемонстрировать их фактическое влияние и практическое использование.

БЛАГОДАРНОСТИ

Авторы хотели бы поблагодарить Маттиаса Шеррера за его

Вклад. Эта работа поддержана Швейцарским национальным фондом. Научный фонд (SNSF), грант № 200021_150142.

ССЫЛКИ

[1] A. Ленк, М. Клемс, Дж. Нимис, С. Тай и Т. Сандхолм, «Что такое Внутри облака? Архитектурная карта облачного ландшафта», в Труды семинара ICSE «Программная инженерия. Проблемы облачных вычислений», 2009 г. стр. 23–31.

[2] К. С. Кандан, В.-С. Ли, Т. Фан и М. Чжоу, «На рубежах «Информация и программное обеспечение как услуги» в сборнике «Новые рубежи в области информации и программного обеспечения как услуг». Нью-Йорк, штат Нью-Йорк, США: Springer, 2011. стр. 283–300.

[3] Б. Шварц, Парадокс выбора: почему меньше значит больше. Нью-Йорк, Нью-Йорк, США: Ecco, 2004.

[4] Дж. Канг и К.М. Сим, «Cloudle: многокритериальный облачный сервис» поисковая система», в трудах конференции IEEE Asia-Pacific Services Comput. 2010, стр. 339–346.

[5] Г. Адомавичус и А. Тужилин, «К следующему поколению Рекомендательные системы: обзор современного уровня техники и возможных расширений», IEEE Trans. Knowl. Data Eng., том 17, № 6, стр. 734–749, июнь 2005 г.

[6] И. Тодоран, «Stakecloud: Передача требований заинтересованных сторон и идентификация ресурсов в облаке», в трудах 20-й конференции IEEE Международная конференция по требованиям к инженерам, 2012, стр. 353–356.

[7] Intel Cloud Finder. (Июнь 2015 г.) [Онлайн]. Доступно: <http://www.intelcloudfinder.com/>

[8] А. Хадже-Хоссейни, Д. Гринвуд, Дж. В. Смит и И. Sommerwille, «Инструментарий для внедрения облачных технологий: поддержка решений по внедрению облачных технологий на предприятии», Программное обеспечение: Практический опыт, т. 42, № 4, стр. 447–465, 2012.

[9] С. Зардари, Р. Бахсун и А. Экарт, «Внедрение облака: определение приоритетов препятствий и тактика устранения препятствий с использованием АНР», в Proc. 29-й симпозиум ACM по прикладным вычислениям, 2014, стр. 1013–1020.

[10] С. Зардари и Р. Бахсун, «Внедрение облака: целеустремленное подход к инженерным требованиям», в материалах 2-го Международного семинара Программное обеспечение англ. Облачные вычисления., 2011, стр. 29–35.

[11] Б. Нусейбех и С. Истербрук, «Инженерия требований: дорожная карта», в Труды конференции Future Softw. Eng., 2000, стр. 35–46.

[12] И. Sommerwille и Г. Котонья, Разработка требований: процессы и методы. Хобокен, Нью-Джерси, США: Wiley, 1998.

[13] А. ван Ламсвеерде, Разработка требований: от системных целей От моделей UML до спецификаций программного обеспечения. Нью-Йорк, штат Нью-Йорк, США: Уайли, 2009.

[14] Б. Нусейбех, С. Истербрук и А. Руссо, «Использование непоследовательности в разработке программного обеспечения», Компьютер, т. 33, № 4, стр. 24–29 апреля 2000 г.

[15] У. Дж. Ллойд, М. Б. Россон и Дж. Д. Артур, «Эффективность методов выявления требований при разработке распределенных требований», в трудах Объединенной международной конференции IEEE по требованиям, 2002 г., стр. 311–318.

[16] Т. Цумаки и Т. Тамаи, «Структура для сопоставления методов выявления требований с характеристиками проекта», Программные процессы: практика улучшения, т. 11, № 5, стр. 505–519, 2006.

[17] И. Тодоран, Н. Сейфф и М. Глинц, «Как поставщики облачных услуг выявляют требования потребителей: исследование девятнадцати компаний», в Трудах 21-й Международной конференции IEEE по требованиям к инженерам, 2013 г., стр. 105–114.

[18] И. Тодоран и М. Глинц, «Поиск требований: тщательный анализ расширенных поисковых запросов для облачных сервисов с нечеткими решетками Галуа», в Трудах 10-го Всемирного конгресса IEEE, 2014 г., стр. 234–241.

[19] К. Денеке, М. Эрне и С.Л. Висмат, Соединения Галуа и их применение. Норвелл, Массачусетс, США: Kluwer, 2004.

[20] М. Эрн, Дж. Козловски, А. Мелтон и Г. Е. Стрекер, «Базовые сведения о связностях Галуа», Апп. Нью-Йоркская академия. наук, том. 704, нет. 1, стр. 103–125, 1993.

[21] Р. Белохлавек, «Нечеткие связи Галуа», Math. Logic Quart., т. 45, № 4, стр. 497–504, 1999.

[22] Б. Гантер, Р. Вилле и К. Франcke, Формальный концептуальный анализ: математические основы. Нью-Йорк, штат Нью-Йорк, США: Springer-Verlag, 1997.

[23] П. Резник и Х. Р. Вариян, «Рекомендательные системы», Commun. ACM, т. 40, № 3, стр. 56–58, 1997.

[24] И. Тодоран Коитц и М. Глинц, «Инструмент StakeCloud: от поисковых запросов потребителей облачных услуг до новых требований к сервисам», в трудах 23-й Международной конференции IEEE по требованиям к инженерам, 2015 г., стр. 284–285.

[25] С. Чакраборти, Н. Нагвани и Л. Дей, «Сравнение производительности инкрементальных алгоритмов k-средних и инкрементальных алгоритмов DBSCAN», Int. J. Comput. Appl., т. 27, № 11, стр. 14–18, 2011.

[26] С. Асвани Кумар и С. Шринивас, «Сокращение концептуальной решетки с использованием нечеткой кластеризации k-средних», Expert Syst. Appl., т. 37, № 3, стр. 2696–2704, 2010.

[27] DE Damian и D. Zowghi, «Проблемы RE в организациях по разработке программного обеспечения с несколькими филиалами», Requirements Eng., т. 8, № 3, стр. 149–160, 2003.

[28] П. Грюнбахер и Н. Сейфф, «Согласование требований», в книге «Разработка и управление требованиями к программному обеспечению». Нью-Йорк, штат Нью-Йорк, США: Springer, 2005, стр. 143–162.

[29] А. Даугулис, «Временные аспекты в проектировании требований: или «У каждой тучи есть серебряная подкладка», Requirements Eng., т. 5, № 3, стр. 137–143, 2000.

[30] Л. Лю, Повторное использование программного обеспечения в эпоху облачных вычислений: выявление целевых требований для повторного использования услуг в облачных вычислениях. Херши, Пенсильвания, США: IGI Global, 2012.

[31] Л. Сан, Х. Донг, Ф. К. Хуссейн, О. К. Хуссейн, Дж. Ма и И. Чжан, «Гибридная нечеткая структура для выбора облачных сервисов», в Трудах Международной конференции IEEE по веб-сервисам, 2014 г., стр. 313–320.

[32] Д. Вильегас и С.М. Саджади, «Сопоставление нефункциональных требований с облачными приложениями», в трудах Международной конференции по программному обеспечению и инженерным знаниям, Eng., 2011, стр. 527–532.

[33] К. Бекерс, С. Фассбендер и Х. Шмидт, «Комплексный метод выявления юридических требований на основе шаблонов, применяемых к примеру облачных вычислений», в Трудах 7-й Международной конференции по доступности, Rel. Security, 2012, стр. 463–472.

[34] К. Бекерс, М. Хейзел, И. Коте, Л. Гёке и С. Гулер, «Выявление требований безопасности на основе структурированных шаблонов для облаков», в трудах 8-й Международной конференции по доступности, Rel. Security, 2013, стр. 465–474.

[35] С. Л. Лим, Д. Кеверсия и А. Финкельштейн, «Stakenet: использование социальных сетей для анализа заинтересованных сторон крупномасштабных программных проектов», в трудах 32-й Международной конференции ACM/IEEE по программному обеспечению, 2010 г., том 1, стр. 295–304.

[36] Т. Туунанен, «Новый взгляд на методы выявления требований», Журнал информационных технологий и теории приложений, т. 5, № 3, стр. 45–62, 2003.

[37] Н. Кукрея и Б. Бём, «Влияние инструментов согласования требований на процесс, основанный на социальных сетях», в Трудах Международной конференции. Программное обеспечение и системные процессы, 2012, стр. 68–72.

[38] Дж. Д. Хербслеб, «Глобальная разработка программного обеспечения: будущее социально-технической координации», в трудах конференции Future Softw. Eng., 2007, стр. 188–198.

[39] Б. Декер, Э. Рас, Дж. Реч, П. Жобер и М. Рит, «Участие заинтересованных сторон в разработке требований на основе вики», IEEE Softw., т. 24, № 2, стр. 28–35, март/апрель 2007 г.

[40] C. Solis и N. Ali, «Распределенное выявление требований с использованием пространственного гипертекстового вики», в трудах 5-й Международной конференции IEEE по глобальному программному обеспечению Eng., 2010, стр. 237–246.

[41] П. Лян, П. Авгеру и В. Клерк, «Обоснование требований для распределенного анализа требований с использованием семантической вики», в трудах 4-й Международной конференции IEEE по глобальному программированию, 2009 г., стр. 388–393.

[42] С. Ломанн, Т. Рихерт, С. Ауэр и Дж. Циглер, «Совместная разработка баз знаний при распределенном выявлении требований», в Трудах GI Conf. Softw. Eng. (семинары), 2008, LNI т. 122, стр. 22–28.

[43] Х.-Л. Янг и Дж.-Х. Тан, «Трехэтапная модель выявления требований для веб-информационных систем», Ind. Manage. Data Syst., т. 103, № 6, стр. 398–409, 2003.

[44] Л. Карлссон, А. Дальстедт, Дж. Натт оч Даг, Б. Реннелл и А. Перссон, «Проблемы проектирования требований, ориентированных на рынок, — исследование, проведенное в ходе отраслевых интервью», в трудах 8-го Международного семинара по требованиям к инженерии: качество найденного программного обеспечения, 2003 г., стр. 101–112.

[45] Н. Абдулла, Й. Сюй и С. Гева, «Интеграция методов совместной фильтрации и поиска для персонализированных рекомендаций по продуктам в Интернете», в Трудах 11-й Международной конференции IEEE по интеллектуальному анализу данных, 2011 г., стр. 711–718.

[46] Т. К. Ли, Й. Парк и Й.-Т. Парк, «Временной подход к эффективным рекомендательным системам с использованием неявной обратной связи», Expert Syst. Appl., т. 34, № 4, стр. 3055–3062, 2008.

[47] Дж. Л. Херлокер, Дж. А. Констан, Л. Г. Тервин и Дж. Т. Ридл, «Оценка систем рекомендаций совместной фильтрации», ACM Trans. Inf. Syst., т. 22, № 1, стр. 5–53, 2004.

[48] П. Берхин, «Обзор методов кластеризации данных», в книге «Группировка многомерных данных», под ред. Дж. Когана, К. Николаса и М. Тебуля, Берлин, Германия: Springer, 2006, стр. 25–71.

[49] А. ван Дёрсен и Т. Кёйперс, «Идентификация объектов с использованием кластерного и концептуального анализа», в трудах 21-й Международной конференции по программному обеспечению и инженерии, 1999, стр. 246–255.

[50] Р. Годен, Р. Миссауи и Х. Алауи, «Алгоритмы постепенного формирования концепций на основе решеток Галуа (концепций)», Comput. Интел., вып. 11, нет. 2, стр. 246–267, 1995.

[51] С. Carpineto и G. Romano, «Концептуальная система кластеризации решеток и ее применение для поиска в процессе просмотра», Mach. Learning, т. 24, № 2, стр. 95–122, 1996.

[52] Н. Пернель, М.-К. Руссе, Х. Солдано и В. Вентос, «Масштаб: система на основе вложенных решеток Галуа для концептуальной кластеризации», Дж. Эксп. Теоретический Артип. Интел., вып. 14, нет. 2/3, стр. 157–187, 2002.



Ирина Тодоран Коиц получила степень бакалавра наук в области компьютерных наук в Политехническом университете Бухареста (Румыния) и степень магистра наук с отличием в области проектирования и инженерии сервисов в Университете Аалто (Хельсинкский технологический университет), Финляндия. В настоящее время она работает над докторской степенью в области компьютерных наук в Цюрихском университете под руководством Мартина Глинца. В сфере ее интересов входит обмен информацией о требованиях к облаку, анализ данных и эмпирические исследования. Она является студентом-членом IEEE.



Мартин Глинц получил степень доктора естественных наук в области компьютерных наук в Рейнско-Вестфальском техническом университете Ахена. Он является профессором и заведующим кафедрой информатики Цюрихского университета. До прихода в Цюрихский университет он 10 лет проработал в промышленности. Его интересы включают разработку требований и программную инженерию, в частности моделирование, валидацию и обеспечение качества, а также обучение программной инженерии. Он является членом компьютерного общества IEEE.